M.Maruthi 192425257
ITA0502
Computer Vision
Slot -C

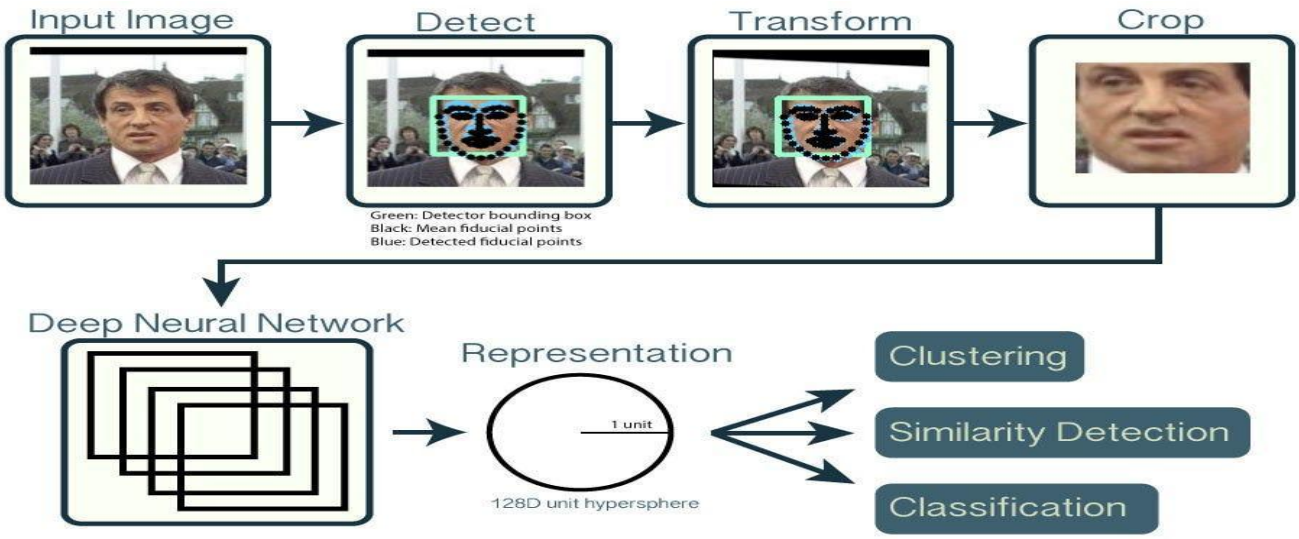
PSEUDOCODE WRITING TASK

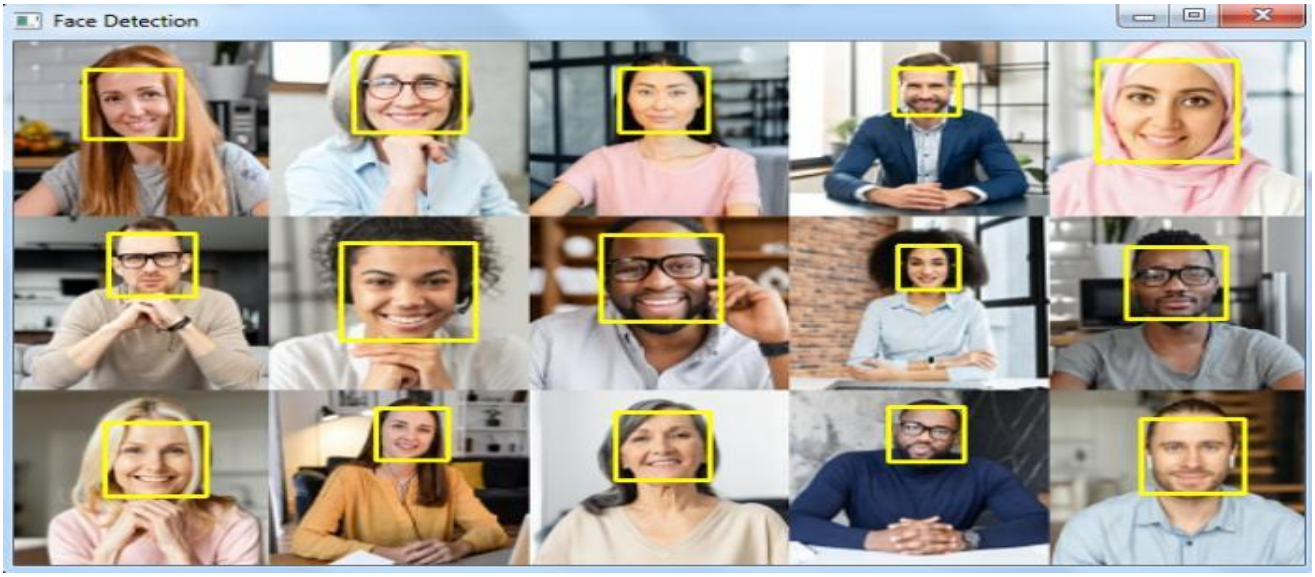
1. Real-Time Face Detection and Recognition

Objective: Design an OpenCV-based system that continuously captures frames from a camera, detects multiple faces, recognizes registered persons, and displays their names with bounding boxes while maintaining real-time performance.

Overall Pipeline:

Camera → Frame Acquisition → Preprocessing → Face Detection → Face Feature Extraction → Recognition → Confidence Check → Visualization → Display





Explanation of Important Steps

1. Image Acquisition

The camera continuously captures frames.

Camera → Frame 1 → Frame 2 → Frame 3 → ...

OpenCV's VideoCapture can be used to access the webcam or CCTV stream.

2. Preprocessing

Each frame can be:

- Resized to reduce computation.
- Converted to grayscale for lightweight face detection.
- Filtered to reduce camera noise.

This reduces processing time and improves detection quality.

3. Multiple Face Detection

The detector searches the complete frame and returns multiple bounding boxes.

For example:

Face 1 → Student A

Face 2 → Student B

Face 3 → Unknown

Face 4 → Student C

The system processes each detected face independently.

4. Feature Extraction

The detected face is converted into useful numerical features.

For a lightweight OpenCV implementation, **LBPH** can be used.

For a more advanced system, facial embeddings can be generated using a deep face-recognition model.

5. Recognition

The extracted features are compared with the trained dataset.

Detected Face
↓
Feature Extraction
↓
Compare with Dataset
↓
Best Match
↓
Confidence Check
↓
Name / Unknown

6. Confidence Threshold

A threshold is important to reduce false recognition.

IF confidence is sufficient

→ Recognized Person

ELSE

→ Unknown

The exact threshold depends on the selected recognition method.

7. Visualization

The final frame displays:

- Face bounding box.
- Person's identity.
- Unknown label when no reliable match exists.

8. Real-Time Optimization

To minimize processing delay:

- Resize input frames.
- Process a smaller frame resolution.
- Skip some frames if necessary.
- Detect faces periodically.

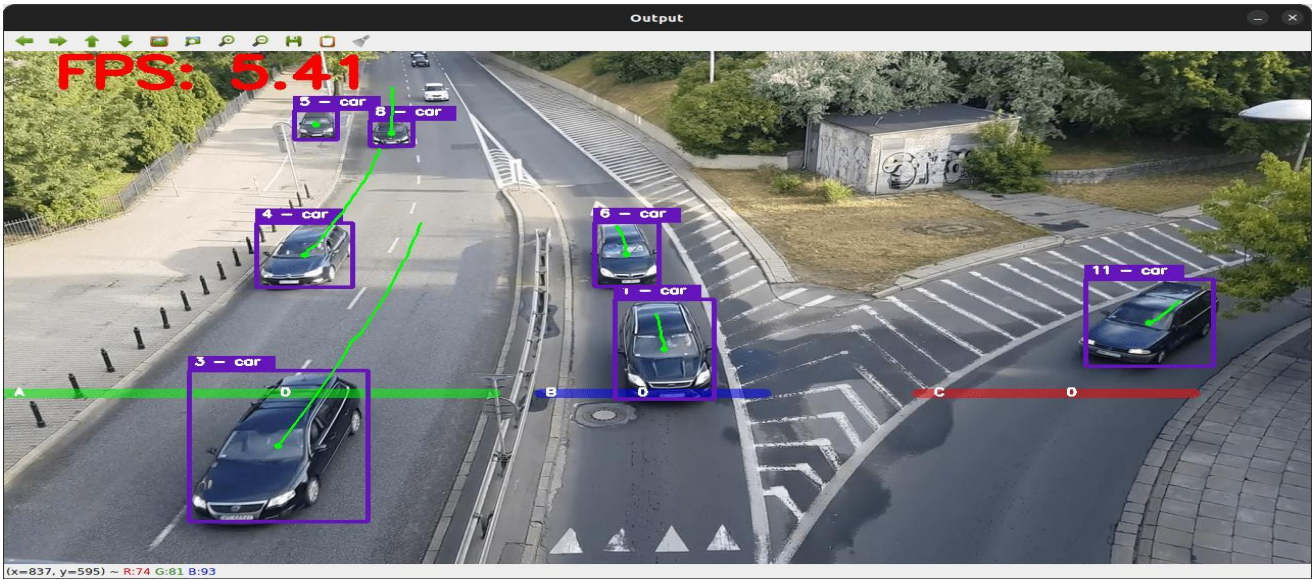
2. Vehicle Detection, Tracking and Counting

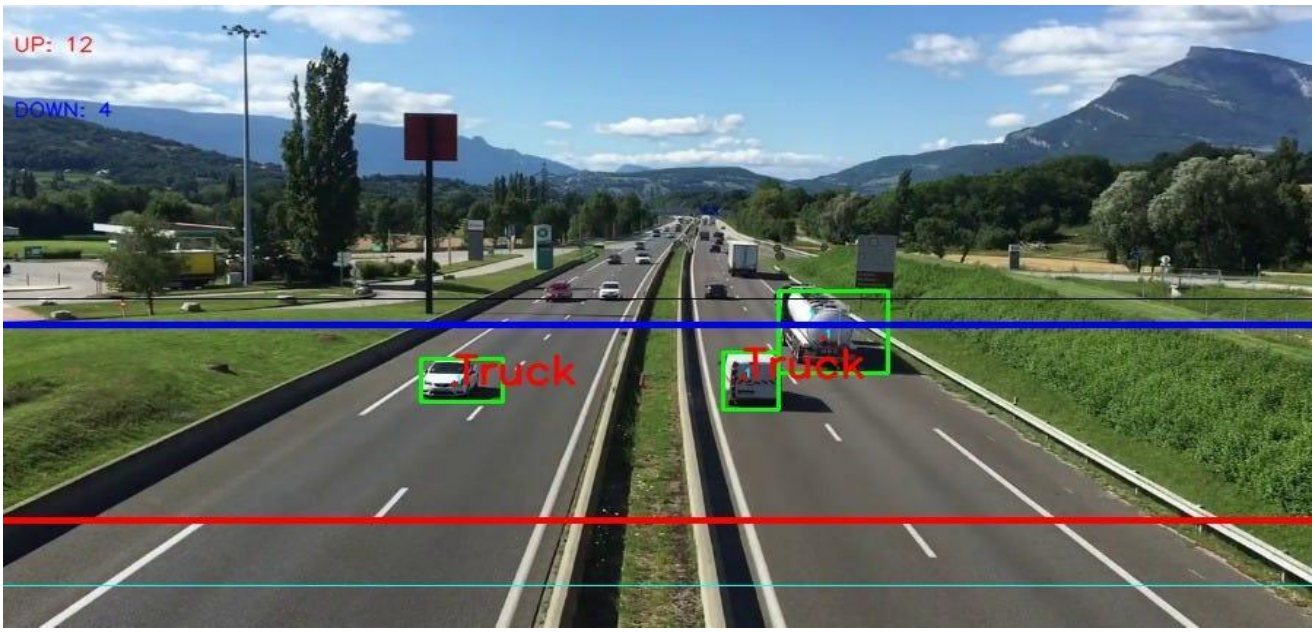
Objective

Design an OpenCV-based traffic monitoring system that detects multiple vehicles, tracks them across frames, counts each vehicle when it crosses a predefined line, and prevents double counting.

Overall Pipeline

Video → Preprocessing → ROI → Vehicle Detection → Tracking → ID Assignment → Line Crossing → Counting → Visualization





1. Video Acquisition

The system continuously reads frames from:

- CCTV camera
- Webcam
- Traffic surveillance camera
- Recorded traffic video

Camera
↓
Frame 1
↓
Frame 2
↓
Frame 3
↓

If the video source cannot be opened, the program terminates safely.

2. Preprocessing

The input frame can be:

- Resized.
- Smoothed using Gaussian/median filtering.
- Restricted to a **Region of Interest**.

The ROI prevents the system from processing irrelevant areas such as buildings, sky, and sidewalks.

3. Vehicle Detection

The detector identifies vehicles inside the ROI.

Possible approaches include:

- Background subtraction + contours for a simple fixed-camera system.
- OpenCV DNN/object detection for more reliable vehicle detection.

The detector returns:

Bounding Box

Vehicle Class

Confidence Score

Example:

Car → Confidence 95%

Bus → Confidence 91%

Truck → Confidence 88%

Low-confidence detections are ignored.

4. Vehicle Tracking

Detection alone is not sufficient because the same vehicle appears in many consecutive frames.

Tracking assigns an ID:

Frame 1 → Car → ID 1

Frame 2 → Car → ID 1

Frame 3 → Car → ID 1

Frame 4 → Car → ID 1

Therefore, the system understands that these detections belong to the **same vehicle**.

6. Line-Crossing Logic

Suppose the counting line is at position Y.

If:

Previous Y < Line Y

AND

Current Y >= Line Y

then the vehicle has crossed the line.

The counter is increased:

vehicle_count = vehicle_count + 1

7. Avoiding Double Counting

This is one of the most important constraints.

Maintain a set:

counted_IDs = {1, 4, 7}

Before increasing the count:

IF vehicle_ID NOT IN counted_IDs

count = count + 1

Add vehicle_ID to counted_IDs

END IF

Therefore:

Vehicle ID 1 crosses line → Count = 1

Vehicle ID 1 appears again → Do NOT count

Vehicle ID 1 appears again → Do NOT count

This prevents duplicate counting.

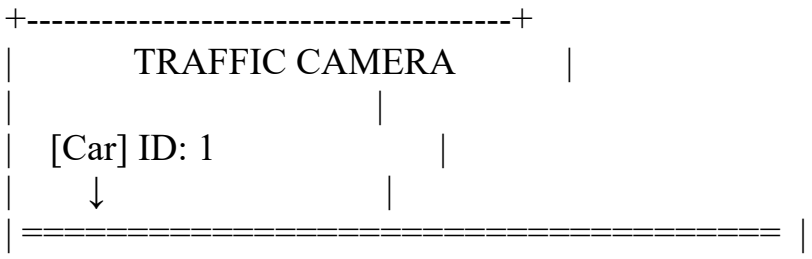
8. Multiple Vehicles

Each vehicle has its own:

- Bounding box
- Tracking ID
- Center position
- Previous position
- Counting status

9. Visualization

The output frame displays:



```
|       COUNTING LINE                |
|                                    |
|   [Truck] ID: 2                    |
|                                    |
| Vehicle Count: 12                  |
+------------------------------------+
```

The following information can be displayed:

- Bounding boxes.
- Vehicle IDs.
- Vehicle classes.
- Virtual line.
- ROI.
- Total vehicle count.

10. Real-Time Optimization

To maintain near real-time performance:

- Resize frames.
- Process only the ROI.
- Use efficient detection models.
- Track vehicles between detection frames.
- Skip unnecessary frames when appropriate.
- Remove expired tracking IDs.
- Avoid repeatedly detecting the entire image.